

A Hardware-in-the-Loop Testbed for Security Analysis of Industrial Communication Protocols

1st Aliaksandra Shuliakouskaya
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
01164310@pw.edu.pl

2nd Krzysztof Sosnowski
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
krzysztof.sosnowski@pw.edu.pl

3rd Tomasz Sokół
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
tomasz.sokol2@pw.edu.pl

4th Waldemar Graniszewski
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
waldemar.graniszewski@pw.edu.pl

Abstract—This paper presents the design and implementation of a Hardware-in-the-Loop (HiL)-based testbed developed for the analysis of security aspects of industrial communication protocols in cyber-physical systems. The proposed environment combines a physical control layer with a simulation layer, enabling experiments to be conducted under conditions that reflect real industrial operation while preserving the flexibility required for controlled testing. The setup utilizes a WAGO programmable logic controller (PLC) connected to a simulation model implemented in MATLAB Simulink, with communication established via the Modbus TCP/IP protocol. Due to its widespread use in industrial automation and its limited built-in security mechanisms, this protocol serves as a relevant case for investigating communication-level vulnerabilities. The developed testbed supports bidirectional data exchange between the physical and simulated components, allowing real-time interaction and observation of system behavior. It also enables acquisition and inspection of network traffic, as well as the introduction of controlled disturbances and modified communication patterns to emulate selected abnormal or potentially malicious scenarios. The architecture of the environment was designed with extensibility in mind, making it possible to adapt the setup to different configurations, protocols, or analysis objectives without significant structural changes. The main goal of the presented work is to provide a practical and reproducible platform for examining the behavior of industrial communication under non-ideal conditions and for identifying potential weaknesses that may affect system reliability and security. The testbed is intended to support further experimental studies, including the evaluation of monitoring approaches, testing of detection mechanisms, and validation of concepts aimed at improving the resilience of industrial control systems against communication-related threats.

Index Terms—Hardware-in-the-Loop, Modbus TCP, industrial control systems, cybersecurity, PLC, SCADA, network security, testbed

I. INTRODUCTION

Industrial control systems (ICS) and cyber-physical systems (CPS) are increasingly exposed to communication-level threats as operational technology (OT) networks become interconnected with corporate and public infrastructure [1], [2].

Legacy industrial protocols, designed primarily for reliability and determinism rather than security, introduce significant vulnerabilities when deployed in modern networked environments.

Modbus TCP/IP is among the most widely deployed protocols in industrial automation. Originally developed for serial communication, it defines no mechanisms for client authentication, session management, or payload encryption [3], [4]. All data – including function codes, register addresses, and process values – are transmitted in plaintext over TCP port 502. As a result, any device with network access to a Modbus server can read process data or issue control commands without restriction. Known attack vectors include unauthorized register reads and writes, command injection, traffic replay, and denial-of-service through connection flooding [2]. The MITRE ATT&CK for ICS framework categorizes these as *Unauthorized Command Message* and *Network Sniffing* techniques [5].

The security implications of these weaknesses have been demonstrated in practice: experimental studies have shown that Modbus TCP-based systems are susceptible to man-in-the-middle manipulation, forged command injection, and covert data exfiltration using only standard network tools [4], [6]. Incident reports further indicate that a significant share of ICS security events involves protocols lacking built-in authentication, underscoring the practical relevance of protocol-level analysis [7].

Testing the security posture of ICS environments is inherently difficult. Experiments on live production systems carry unacceptable operational risk, while pure software simulation fails to capture the real-time behavior, timing characteristics, and hardware-specific responses of physical controllers. Hardware-in-the-Loop (HiL) simulation addresses this gap by coupling physical hardware with software models, enabling realistic yet controlled experimentation [6].

This paper presents a low-cost, open HiL testbed built

around a WAGO PLC and MATLAB/Simulink, targeted at the experimental analysis of Modbus TCP/IP security properties. The testbed provides a reproducible platform for investigating communication-level vulnerabilities without risk to real infrastructure, and is designed to be extended to additional protocols and controller configurations in future work.

The main contributions are: (1) a practical HiL testbed architecture combining a physical PLC with real-time simulation; (2) quantitative characterization of Modbus TCP/IP latency under normal and disturbed conditions; and (3) experimental demonstration of attack scenarios exploiting the absence of authentication and encryption in the protocol.

II. TESTBED ARCHITECTURE

A. Overview

The testbed consists of two interconnected layers illustrated in Fig. 1. The *physical control layer* is a WAGO PLC programmed in CODESYS 3.5. The *simulation layer* is a MATLAB/Simulink model acting as Modbus TCP master. Both layers communicate over a standard Ethernet link on TCP port 502. A Wireshark instance running on the simulation host passively captures all traffic for offline analysis.

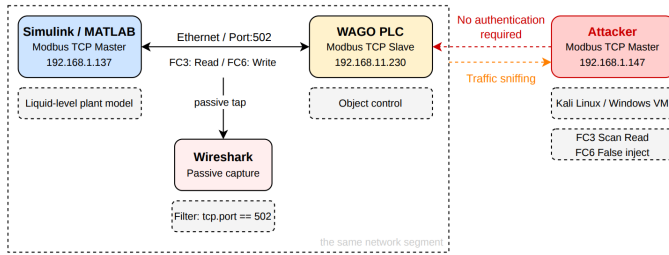


Fig. 1. HiL testbed architecture. Simulink/MATLAB acts as Modbus TCP master; the WAGO PLC operates as slave. Wireshark passively captures all traffic on port 502. An attacker machine on other network segment connects as an unauthorized Modbus TCP master, issuing FC6 write and FC3 scan requests without authentication.

B. Simulation Layer – Plant Model

The simulation layer is implemented in MATLAB/Simulink and acts as both the physical plant model and the Modbus TCP master. It simulates a liquid tank filling process governed by the volumetric flow balance:

$$\frac{dV}{dt} = \frac{1}{A}(Q_{in} - Q_{out}) \quad (1)$$

where A is the tank cross-sectional area, $Q_{in} = K_p \cdot u$ is the inflow rate proportional to the binary control signal $u \in \{0, 1\}$ received from the PLC, with K_p representing the maximum flow rate. The outflow follows Torricelli's law:

$$Q_{out} = C_d \cdot A_o \cdot \sqrt{2gh} \quad (2)$$

where C_d is the discharge coefficient, A_o is the outlet orifice area, $h = V/A$ is the current liquid level, and $g = 9.81 \text{ m/s}^2$. The model integrates equations (1)–(2) at each 100 ms step and writes the resulting liquid level h to a PLC holding register via Modbus TCP FC6. Separate MATLAB scripts execute latency

measurement, disturbance injection, and attack emulation, each via an independent TCP connection concurrent with the Simulink model.

C. Physical Control Layer

The physical control layer is a WAGO PLC programmed in CODESYS 3.5. Each control cycle the PLC reads the liquid level h from the holding register (%QW0) written by Simulink, and applies a two-position control law: the filling valve is opened ($u = 1$) when the level falls below the lower threshold, and closed ($u = 0$) when it exceeds the upper threshold. The resulting binary control signal u is written back to a holding register (%QW1), which Simulink reads to update Q_{in} in the next integration step, closing the HiL loop. A status flag mapped to a discrete coil (%QX0.0) indicates whether the tank is within safe operating bounds. The Modbus TCP slave is configured on port 502 with ten holding registers and sixteen coils, with the *Writeable* flag enabled.

D. Traffic Capture

Wireshark was deployed on the simulation host with a capture filter restricted to TCP port 502. Captured traffic confirmed that all Modbus TCP frames – including function codes, register addresses, and process values – are transmitted in plaintext with no transport-layer security. Fig. 2 shows a representative dissected frame: Function Code 6 (Write Single Register) with the liquid-level value visible as a 16-bit unsigned integer. This absence of encryption means any observer with passive network access can reconstruct the full process state in real time.

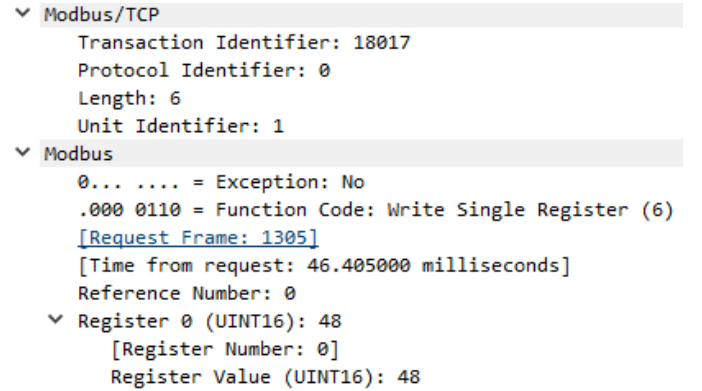


Fig. 2. Wireshark dissection of a Modbus TCP frame. Function code, register address, and process value are all transmitted in plaintext.

III. EXPERIMENTAL EVALUATION

A. Baseline Characterization

To establish a timing reference for all subsequent experiments, 500 round-trip latency measurements were collected during normal operation at a 100 ms sampling interval. Each measurement covered one full FC6 write followed by an FC3 read of the same register.

The mean round-trip latency was 49.00 ms ($\sigma = 2.57 \text{ ms}$) with a maximum of 96.24 ms and no packet losses (Table I,

Fig. 3). The latency distribution is approximately normal, consistent with a well-behaved TCP/IP stack under light load. However, the maximum value approaching the 100 ms control cycle period indicates limited timing headroom: any additional network load or deliberate interference risks saturating the communication window and causing missed control updates.

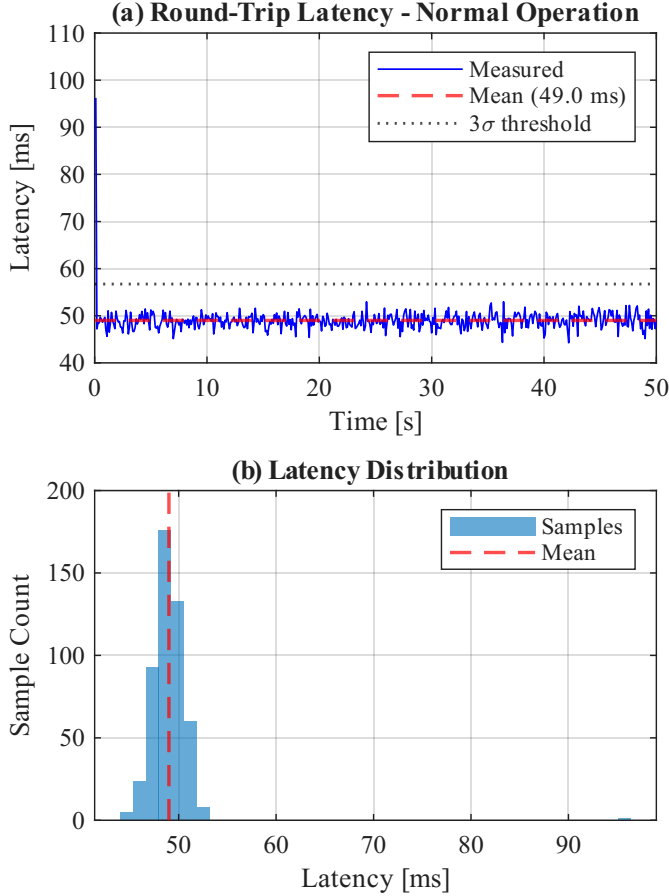


Fig. 3. Baseline characterization: (top) round-trip latency over time with mean and 3σ threshold; (bottom) latency distribution ($n = 500$, mean = 49.00 ms, $\sigma = 2.57$ ms).

B. Disturbance Experiments

Two disturbance scenarios were applied to characterize protocol sensitivity to non-ideal network conditions. Results are shown in Fig. 4 and Table I.

Experiment 1 – Artificial Packet Delay. Random transmission delays uniformly distributed over $[0, 200]$ ms were injected before each write. Mean latency rose to 98.54 ms ($\sigma = 3.02$ ms), effectively saturating the 100 ms control window and consistently exceeding the baseline 3σ threshold. Under these conditions the master cannot dispatch the next control update before the previous transaction completes, causing cumulative desynchronization of the liquid-level control loop.

Experiment 2 – Communication Timeout. Three blackout windows of 20 samples each were introduced by suspending all transmissions. During active periods latency was 86.29 ms ($\sigma = 32.82$ ms), with the high standard deviation reflecting

post-blackout recovery transients. Critically, the WAGO PLC exhibited no autonomous detection or alarming during communication interruptions, silently retaining its last register values. An attacker exploiting this behavior could suspend legitimate traffic and substitute arbitrary process data upon reconnection without triggering any PLC-side alert.

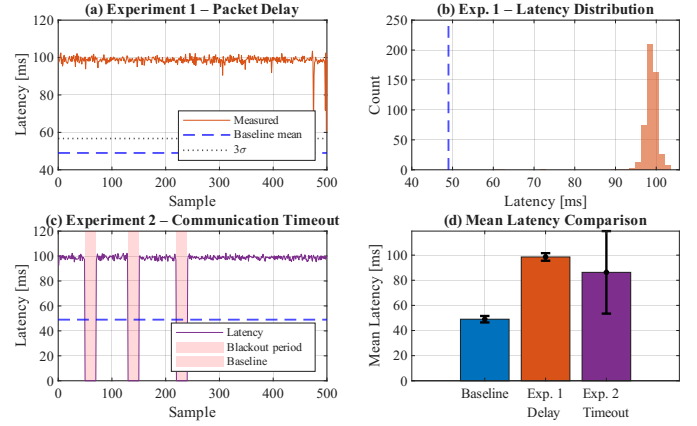


Fig. 4. Disturbance experiments: (top) Exp.1 packet delay saturates the 100 ms control window; (bottom) Exp.2 communication timeout with blackout windows highlighted – no PLC alarm is triggered during any interruption.

C. Attack Scenarios

Attack 1 – Unauthorized Register Write. A second MATLAB client established an independent Modbus TCP connection to the PLC while the Simulink model was running. No authentication challenge was issued by the PLC. The attacker successfully overwrote the liquid-level holding register (%QW0) with an injected value of 0 across 100 consecutive samples (Fig. 5, top). Since the WAGO PLC reads this register each cycle to decide whether to open or close the filling valve, the injected zero caused the controller to read a falsified level and hold the valve open continuously, regardless of the actual level computed by the Simulink plant model. The legitimate Simulink master received no notification of the concurrent connection or of the register modification. Mean attacker round-trip latency was 74.35 ms ($\sigma = 24.97$ ms), elevated relative to baseline during concurrent execution of the MATLAB/Simulink environment and the MATLAB-based attack scripts. To verify whether the observed timing effects originated from network contention or from the attack implementation itself, supplementary experiments were conducted using an external attacker host running Kali Linux and an independent Windows-based traffic generator injecting false process values. In this configuration, successful unauthorized register manipulation was still achieved, however no significant latency increase relative to baseline operation was observed.

Attack 2 – Register Scanning. Sequential FC03 (Read Holding Registers) requests were issued to all configured register addresses without any prior knowledge of the PLC data model. All registers responded successfully, exposing non-zero process values at addresses 1 and 4 – corresponding to the

scaled liquid level and the binary tank-status flag respectively. The zero response-time standard deviation ($\sigma = 0.00$ ms) confirms fully deterministic server behavior: the PLC applies no rate-limiting, throttling, or anomaly response to repeated sequential queries from an unknown client, enabling complete data model enumeration in under one second (Fig. 5, bottom).

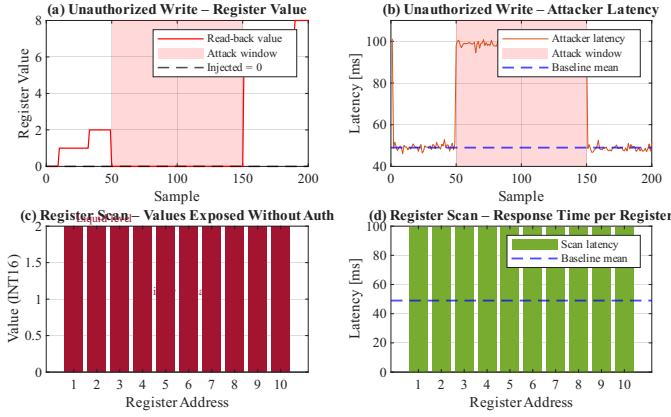


Fig. 5. Attack scenarios: (top) unauthorized write forces the liquid-level register to zero throughout the attack window; (bottom) register scan exposes all non-zero holding register values without authentication.

D. Summary of Results

TABLE I
SUMMARY OF EXPERIMENTAL RESULTS

Scenario	Mean [ms]	Std [ms]	Max [ms]	Lost [%]
Baseline (normal)	49.00	2.57	96.24	0.0
Exp. 1 – Packet Delay	98.54	3.02	103.45	0.0
Exp. 2 – Timeout	86.29	32.82	102.37	0.0
Attack – Unauth. Write	74.35	24.97	102.93	0.0
Attack – Reg. Scan	99.47	0.00	99.47	0.0

The results demonstrate three exploitable properties of standard Modbus TCP/IP. First, the complete absence of authentication permits any network client to connect and issue read or write commands undetected. Second, plaintext transmission exposes the full process state to passive observers with no decryption required. Third, the protocol operates with limited timing headroom at a 100 ms cycle, making it susceptible to performance degradation under deliberate delay injection and communication disturbances. Follow-up validation experiments further showed that latency increases observed during unauthorized write attacks were strongly dependent on the software implementation of the attacking client and measurement environment. Therefore, timing anomalies should be interpreted cautiously and not treated as universal indicators of malicious activity [8], [9].

IV. CONCLUSION

This paper presented a low-cost, reproducible Hardware-in-the-Loop testbed for security analysis of industrial communication protocols. A MATLAB/Simulink model of a liquid

tank filling process – governed by a volumetric flow balance with Torricelli outflow – was coupled with a WAGO PLC acting as the two-position level controller, communicating via Modbus TCP/IP. Baseline characterization, disturbance injection, and attack emulation experiments confirmed that Modbus TCP/IP in standard configuration provides no resistance to unauthorized access, process data exposure, or timing-based disruption. All attack scenarios succeeded without authentication and without triggering any PLC-side detection. Supplementary validation experiments additionally indicated that timing-related effects observed during attack execution were influenced by the software environment used for traffic generation and measurement. This highlights the importance of carefully separating protocol-level behavior from implementation-dependent artifacts in experimental ICS cybersecurity research. The testbed architecture is extensible to alternative industrial protocols and controller platforms, supporting future comparative security studies across different protocol generations and ICS configurations [6].

REFERENCES

- [1] K. Stouffer, J. Falco, and K. Scarfone, “Guide to industrial control systems (ICS) security,” National Institute of Standards and Technology, Tech. Rep. Special Publication 800-82, Revision 2, 2015.
- [2] W. Alsabbagh and P. Langendörfer, “Security of programmable logic controllers and related systems: Today and tomorrow,” *IEEE Open Journal of the Industrial Electronics Society*, vol. 4, pp. 659–693, 2023.
- [3] Modbus Organization, *Modbus Application Protocol Specification V1.1b3*, 2012. [Online]. Available: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf
- [4] F. Katulić, D. Sumina, S. Groš, and I. Erceg, “Protecting Modbus/TCP-based industrial automation and control systems using message authentication codes,” *IEEE Access*, vol. 11, pp. 47 007–47 023, 2023.
- [5] MITRE Corporation, “ATT&CK for industrial control systems,” <https://attack.mitre.org/matrices/ics/>, 2023.
- [6] I. B. de Brito and R. T. de Sousa, “Development of an open-source testbed based on the Modbus protocol for cybersecurity analysis of nuclear power plants,” *Applied Sciences*, vol. 12, no. 16, p. 7942, 2022.
- [7] Dragos Inc., “OT/ICS cybersecurity year in review 2025 (9th annual report),” Dragos, Tech. Rep., 2026. [Online]. Available: <https://www.dragos.com/ot-cybersecurity-year-in-review/>
- [8] N. Goldenberg and A. Wool, “Accurate modeling of Modbus/TCP for intrusion detection in SCADA systems,” *International Journal of Critical Infrastructure Protection*, vol. 6, no. 2, pp. 63–75, 2013.
- [9] T. Ghosh, S. Bagui, S. Bagui, M. Kadzis, and J. Bare, “Anomaly detection for Modbus over TCP in control systems using entropy and classification-based analysis,” *Journal of Cybersecurity and Privacy*, vol. 3, no. 4, pp. 895–913, 2023.